
ECE251C Final Project Proposal

Evan Dong, Hoseon Yu

March 25, 2026

1 Architecture

The architecture that has been chosen for this project is a 16-bit CPU inspired by the MIPS and Intel 8080 processors. The goal of the project is to build a CPU and a compact ISA and implement it in SystemVerilog. It will have 16 registers and 16-bit-width instructions with 3 different types: R, I, and J.

2 Design Choices

The logic will heavily rely on an accumulator register because of the small bit width. R-type instructions require the use of two reference registers(rs and rt) as operands and the storing of the result into either one of the operand registers or a separate result register(rd). For I types, there needs to be a destination register and a reference register, as well as an immediate. Given that our system uses 16 registers with 16 bit instructions the, including the registers in the instructions would be difficult: 16 registers means that addressing the registers would take 4 bits each and incorporating the same rs, rt, and rd behavior in MIPS would leave no room for a function code in the R type instructions and only a 4 bit immediate for the I type instructions. These complications would lead to a variety of issues. No function code would limit the amount of functions that could be performed to just 16 operations based on the op code, and having only a 4 bit immediate in the I type would constrict the range that base register addressing could move with one beq operation—meaning more code would need to be written to jump large distances. The solution to both of these problems is to implicitly set the destination register to an accumulator register for each operation—freeing up 4 bits in each instruction type.

R-Type:

4bit opcode	4bit rt	4bit rs	4bit funct
-------------	---------	---------	------------

I-Type:

4bit opcode	4bit rd	8bit immediate
-------------	---------	----------------

J-Type:

4bit opcode	12bit address
-------------	---------------

Memory Pointer registers will also be implemented so that there would be no need to specify registers in the I-type instruction to load and save words to memory. Like the accumulator for PC-relative addressing, the load and store instructions would imply the use of these registers. Two registers were implemented to increase the reach of these operations. To make arithmetic operations easier, a flag register will also be implemented to simplify arithmetic operations and keep track of what the state of the current accumulator register is: whether it overflowed, needs a carry, or is negative. This will simplify how arithmetic operations are performed in 16-bit registers.

Registers		Registers	
0	Accumulator	8	Gen
1	Gen	9	Mem pointer
2	Gen	10	Mem pointer
3	Gen	11	SP
4	Gen	12	Flags
5	Gen	13	Reserved
6	Gen	14	Reserved
7	Gen	15	Reserved

Table 1: Registers